

山东大学



实验三实验报告 Experiment report

组员 1：熊梓刚、组员 2：高焜、组员 3：马旭恩、组员 4：陈奕顺

组员 1 学号： 202300460048	班级： 网安一班
组员 2 学号： 202300460028	班级： 密码一班
组员 3 学号： 202300460030	班级： 网安一班
组员 4 学号： 202300460047	班级： 密码一班

声明

本实验主要由熊梓刚、高焜、马旭恩、陈奕顺完成。

熊梓刚完成的部分：T-table 优化、AES-NI 硬件加速、性能测试与分析、图表绘制实验总结；

高焜完成的部分：S-box 基础实现（含密钥扩展）、CTR 模式、标量调试；

马旭恩完成的部分：SIMD Shuffle 优化、XTS 模式（含密文挪用）；

陈奕顺完成的部分：GCM 模式（含 GHASH）、算法背景介绍。

2026 年 7 月 24 日

1 实验背景与目标

1.1 对称密码算法概述

对称密码算法是密码学中最基础、应用最广泛的密码体制。其核心特征是加密和解密使用相同的密钥。AES 由美国 NIST 于 2001 年发布 (FIPS 197)，用于取代老化的 DES 算法，是目前全球使用最广泛的对称分组密码算法。

AES 基于 Rijndael 算法设计，具有以下基本特征：

- **分组长度**：128 位 (16 字节)，表示为 4×4 的字节矩阵
- **密钥长度**：128/192/256 位 (分别对应 AES-128/192/256)
- **轮数**：10/12/14 轮 (由密钥长度决定)
- **结构**：SPN (Substitution-Permutation Network, 代换-置换网络)

1.2 实验目标

本实验旨在从基础实现出发，逐步优化 AES 对称密码算法的软件执行效率。具体要求包括：

1. 实现 AES 的基本 S-box 查表版本 (基准实现)
2. 实现 T-table 优化方案 (预计算合并查找表)
3. 实现 SIMD Shuffle 优化方案 (基于 SSSE3 的 PSHUFB 指令)
4. 实现 AES-NI 硬件加速方案 (Intel AES 新指令)
5. 覆盖 CTR、GCM、XTS 三种工作模式

通过对比不同优化方法的执行效率，理解对称密码算法软件优化的核心思想和实现技术。

1.3 AES 算法结构

AES 加密过程由以下四个基本操作组成 (解密为其逆操作)：

1. **SubBytes** (字节代换)：使用 16×16 的 S-box 对每个字节进行非线性替换
2. **ShiftRows** (行移位)：将状态矩阵的第 i 行循环左移 i 个字节
3. **MixColumns** (列混合)：将每一列视为 $GF(2^8)$ 上的多项式，与固定多项式相乘
4. **AddRoundKey** (轮密钥加)：状态与轮密钥按位异或

加密流程为：初始 AddRoundKey $\rightarrow$ 9/11/13 轮 (SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ AddRoundKey) $\rightarrow$ 最后一轮 (SubBytes $\rightarrow$ ShiftRows $\rightarrow$ AddRoundKey)。

2 基础实现：S-box 查表法

2.1 设计思路

S-box 查表法是 AES 最直观、最基础的软件实现方式。每一轮依次执行 SubBytes、ShiftRows、MixColumns 和 AddRoundKey 四个步骤，每个步骤独立完成。这种方法代码清晰、易于理解，是优化的基准 (Baseline)。

2.2 核心数据结构

AES 状态 (State) 以列优先 (Column-Major) 方式存储为 4 个 32 位字:

Listing 1: AES 状态与密钥结构

```
1 typedef struct {
2     uint32_t rk[AES_MAX_RK_WORDS]; /* 扩展轮密钥: 4*(rounds+1) 个32位字 */
3     int      rounds;                /* 轮数: 10/12/14 */
4 } AES_Key;
```

2.3 密钥扩展

密钥扩展算法将原始密钥扩展为轮密钥序列。AES-128 需要 44 个字 (11 轮 $\times$ 4 字), AES-192 需要 52 个字, AES-256 需要 60 个字。

Listing 2: S-box 密钥扩展 (适用于所有标量方法)

```
1 static int aes_sbox_set_encrypt_key(AES_Key *ctx, const uint8_t *key, int key_len)
2 {
3     const int Nk = key_len / 4; /* 密钥字数 (4/6/8) */
4     ctx->rounds = Nk + 6;       /* 轮数 (10/12/14) */
5     uint32_t *rk = ctx->rk;
6     for (int i = 0; i < Nk; i++)
7         rk[i] = LOAD_U32_BE(key + 4 * i); /* 大端序加载 */
8
9     for (int i = Nk; i < 4 * (ctx->rounds + 1); i++) {
10         uint32_t t = rk[i - 1];
11         if ((i % Nk) == 0) {
12             t = AES_SUB_WORD(ROTL32(t, 8)) ^ AES_RCON[i / Nk - 1];
13         } else if (Nk > 6 && (i % Nk) == 4) {
14             t = AES_SUB_WORD(t);
15         }
16         rk[i] = rk[i - Nk] ^ t;
17     }
18     return 0;
19 }
```

2.4 轮函数实现

每个基本操作独立实现为标量函数:

Listing 3: SubBytes: 逐字节 S-box 查表

```
1 static void sub_bytes(uint32_t *s)
2 {
3     for (int i = 0; i < 4; i++)
4         s[i] = AES_SUB_WORD(s[i]); /* 对32位字的每字节执行S-box */
5 }
6 /* AES_SUB_WORD 宏: 对32位字的4个字节分别查S-box并重组 */
7 #define AES_SUB_WORD(t) \
```

```
8 ((AES_SBOX[(t) >> 24]) << 24) | \  
9 (AES_SBOX[(t) >> 16] & 0xFF) << 16) | \  
10 (AES_SBOX[(t) >> 8] & 0xFF) << 8) | \  
11 (AES_SBOX[(t) & 0xFF]))
```

Listing 4: ShiftRows: 字节级旋转

```
1 static void shift_rows(uint32_t *s)  
2 {  
3     uint32_t t[4];  
4     for (int i = 0; i < 4; i++) t[i] = s[i];  
5     s[0] = (t[0] & 0xFF000000) | (t[1] & 0x00FF0000) |  
6           (t[2] & 0x0000FF00) | (t[3] & 0x000000FF);  
7     s[1] = (t[1] & 0xFF000000) | (t[2] & 0x00FF0000) |  
8           (t[3] & 0x0000FF00) | (t[0] & 0x000000FF);  
9     /* ... s[2], s[3] 类似 */  
10 }
```

Listing 5: MixColumns: $GF(2^8)$ 域矩阵乘法

```
1 static void mix_columns(uint32_t *s, int encrypt)  
2 {  
3     uint8_t a[4][4], b[4][4];    /* a[col][row] */  
4     for (int i = 0; i < 4; i++)  
5         for (int j = 0; j < 4; j++)  
6             a[i][j] = (uint8_t)(s[i] >> (24 - 8 * j));  
7  
8     for (int c = 0; c < 4; c++) {  
9         if (encrypt) {  
10             /* MixColumns: [2 3 1 1; 1 2 3 1; 1 1 2 3; 3 1 1 2] */  
11             b[0][c] = xtime(a[c][0]) ^ (xtime(a[c][1]) ^ a[c][1])  
12                   ^ a[c][2] ^ a[c][3];  
13             /* ... b[1][c], b[2][c], b[3][c] */  
14         } else {  
15             /* InvMixColumns: [0e 0b 0d 09; 09 0e 0b 0d; ...] */  
16             b[0][c] = gf_mul(a[c][0], 0x0e) ^ gf_mul(a[c][1], 0x0b)  
17                   ^ gf_mul(a[c][2], 0x0d) ^ gf_mul(a[c][3], 0x09);  
18             /* ... */  
19         }  
20     }  
21     /* 存储结果: s[col] = [b[0][col] | b[1][col] | b[2][col] | b[3][col]] */  
22     for (int i = 0; i < 4; i++)  
23         s[i] = ((uint32_t)b[0][i] << 24) | ((uint32_t)b[1][i] << 16) |  
24               ((uint32_t)b[2][i] << 8) | ((uint32_t)b[3][i]);  
25 }
```

2.5 性能分析

S-box 查表法的优点在于实现简单、代码清晰。但每轮需要进行：

- 16 次 S-box 查表 (SubBytes)
- 复杂的位操作 (ShiftRows)
- $GF(2^8)$ 乘法 (MixColumns), 包括 `xtime` 和 `gf_mul` 函数调用

这些操作均为标量运算, 无法利用现代 CPU 的 SIMD 并行能力, 因此性能最低 (约 63 MB/s), 作为优化的基准线。

3 T-table 优化方法

3.1 优化原理

T-table 方法由 Daemen 和 Rijmen 在 AES 原始设计中提出。其核心思想是将 SubBytes、ShiftRows 和 MixColumns 三个步骤合并为四次 256-entry 的 32 位表查找和四次 XOR 操作。

3.1.1 数学推导

设状态矩阵的列为 s_0, s_1, s_2, s_3 (每个 s_i 为 32 位字, 代表一列)。一轮变换的输出列 s'_0 为:

$$\begin{aligned} s'_0 &= \begin{bmatrix} 02 & 03 & 01 & 01 \end{bmatrix} \begin{bmatrix} S[s_{0,0}] & S[s_{1,1}] & S[s_{2,2}] & S[s_{3,3}] \end{bmatrix}^T \oplus k'_0 \\ &= Te_0[s_{0,0}] \oplus Te_1[s_{1,1}] \oplus Te_2[s_{2,2}] \oplus Te_3[s_{3,3}] \oplus k'_0 \end{aligned} \quad (1)$$

其中四个 T-table 定义为 (每个 256 项 $\times$ 4 字节 = 1KB):

$$Te_0[x] = \{02 \cdot S[x], 01 \cdot S[x], 01 \cdot S[x], 03 \cdot S[x]\} \quad (2)$$

$$Te_1[x] = \{03 \cdot S[x], 02 \cdot S[x], 01 \cdot S[x], 01 \cdot S[x]\} \quad (3)$$

$$Te_2[x] = \{01 \cdot S[x], 03 \cdot S[x], 02 \cdot S[x], 01 \cdot S[x]\} \quad (4)$$

$$Te_3[x] = \{01 \cdot S[x], 01 \cdot S[x], 03 \cdot S[x], 02 \cdot S[x]\} \quad (5)$$

3.2 核心实现

Listing 6: T-table 加密轮函数

```
1 void AES_TB0X_EncryptBlock(const AES_Key *ctx, const uint8_t *in, uint8_t *out)
2 {
3     const uint32_t *rk = ctx->rk;
4     uint32_t s0, s1, s2, s3, t0, t1, t2, t3;
5
6     /* 初始AddRoundKey */
7     s0 = LOAD_U32_BE(in + 0) ^ rk[0];
8     s1 = LOAD_U32_BE(in + 4) ^ rk[1];
9     s2 = LOAD_U32_BE(in + 8) ^ rk[2];
10    s3 = LOAD_U32_BE(in + 12) ^ rk[3];
11
12    /* 使用T-table的9/11/13轮 */
13    for (int r = 1; r < ctx->rounds; r++) {
14        t0 = AES_TE0[(s0 >> 24) ] ^ AES_TE1[(s1 >> 16) & 0xFF] ^
15            AES_TE2[(s2 >> 8) & 0xFF] ^ AES_TE3[ s3 & 0xFF]
```

```

16     ^ rk[4 * r + 0];
17     t1 = AES_TE0[(s1 >> 24) ] ^ AES_TE1[(s2 >> 16) & 0xFF] ^
18         AES_TE2[(s3 >> 8) & 0xFF] ^ AES_TE3[ s0  & 0xFF]
19     ^ rk[4 * r + 1];
20     /* ... t2, t3 类似 */
21     s0 = t0; s1 = t1; s2 = t2; s3 = t3;
22 }
23
24 /* 最后一轮: 仅SubBytes+ShiftRows (不含MixColumns) */
25 rk += ctx->rounds * 4;
26 s0 = (AES_SBOX[(t0 >> 24) ] << 24) |
27     (AES_SBOX[(t1 >> 16) & 0xFF] << 16) | /* ... */
28     rk[0];
29 /* ... 存储输出 */
30 }

```

3.3 性能分析

T-table 方法将每轮的计算量从: 16 次 S-box 查表 + 位旋转 + GF 乘法 + 16 次 XOR 简化为 16 次 T-table 查表 + 16 次 XOR。

实测性能约 334 MB/s, 相比 S-box 方法 (63 MB/s) 提升了约 **5.3 倍**。代价是额外占用 8KB 内存 (4 张加密表 + 4 张解密表)。

4 SIMD Shuffle 优化方法

4.1 优化原理

SSSE3 指令集引入了 PSHUFB 指令, 可以在单个指令周期内完成 16 字节的任意排列。利用这一特性, AES 的 ShiftRows 操作可以由一条 PSHUFB 指令完成, 消除了标量实现中的复杂位掩码和移位操作。

此外, SSE2 的 128 位 XOR 指令可以一次处理整个 16 字节的状态, 加速 AddRoundKey 操作。

4.2 PSHUFB 的 ShiftRows 实现

ShiftRows 操作在列优先存储下的字节映射关系如表1所示。

表 1: ShiftRows 与 InvShiftRows 的 PSHUFB 掩码

位置	ShiftRows (加密)				InvShiftRows (解密)			
	0	1	2	3	0	1	2	3
列 0	0	5	10	15	0	13	10	7
列 1	4	9	14	3	4	1	14	11
列 2	8	13	2	7	8	5	2	15
列 3	12	1	6	11	12	9	6	3

掩码含义：目标字节 i = 源字节 $\text{mask}[i]$ 。例如加密时，目标位置 0 的字节来自源位置 0（不变），位置 1 的字节来自源位置 5（行 1 循环左移 1），依此类推。

Listing 7: PSHUFB ShiftRows 实现

```
1 static void shuffle_shift_rows(uint8_t *state)
2 {
3     __m128i s = _mm_loadu_si128((const __m128i *)state);
4     s = _mm_shuffle_epi8(s, _mm_load_si128((const __m128i *)SR_MASK));
5     _mm_storeu_si128((__m128i *)state, s);
6 }
```

4.3 SSE2 AddRoundKey

AddRoundKey 由一条 SSE2 XOR 指令完成：

Listing 8: SSE2 AddRoundKey

```
1 static void shuffle_add_round_key(uint32_t *s, const uint32_t *rk, int round)
2 {
3     __m128i ss = _mm_loadu_si128((const __m128i *)s);
4     __m128i rr = _mm_loadu_si128((const __m128i *)rk + 4 * round);
5     ss = _mm_xor_si128(ss, rr);
6     _mm_storeu_si128((__m128i *)s, ss);
7 }
```

4.4 性能讨论

实验结果表明，Shuffle 方法的 ECB 模式性能（约 22 MB/s）反而低于标量 S-box 方法（63 MB/s）。原因分析如下：

1. **数据格式转换开销**：当前实现中，SubBytes 和 MixColumns 仍使用标量的 `uint32_t` 表示。每次 ShiftRows 需要将 `uint32_t[4]` 转为字节数组，执行 PSHUFB 后再转回 `uint32_t[4]`。这种格式转换引入了显著的往返开销。
2. **优化的局限性**：仅优化了 ShiftRows 和 AddRoundKey，SubBytes 和 MixColumns 仍为标量运算，限制了整体加速效果。
3. **全 SIMD 化方向**：要达到显著的性能提升，需要将整个轮函数（包括 SubBytes 和 MixColumns）全部使用 SIMD 实现。这通常需要采用位切片技术，将 16 个 AES 状态按位重新排列，使 SIMD 寄存器同时处理多个数据块的不同位。

尽管如此，Shuffle 方法作为 T-table 和 AES-NI 之间的中间方案，展示了 SIMD 指令在对称密码优化中的应用潜力。在 CTR 和 GCM 模式中，由于工作模式的并行特性掩盖了块加密的开销，所有方法的吞吐量相近（约 275 MB/s）。

5 AES-NI 硬件加速方法

5.1 指令集概述

Intel 于 2010 年在 Westmere 架构中引入了 AES New Instructions (AES-NI)，提供了以下专用指令：

表 2: AES-NI 指令集

指令	功能	说明
<code>aesenc</code>	一轮 AES 加密	SubBytes + ShiftRows + MixColumns + AddRoundKey
<code>aesenclast</code>	最后一轮 AES 加密	SubBytes + ShiftRows + AddRoundKey (无 MixColumns)
<code>aesdec</code>	一轮 AES 解密	InvSubBytes + InvShiftRows + InvMixColumns + AddRoundKey
<code>aesdeclast</code>	最后一轮 AES 解密	InvSubBytes + InvShiftRows + AddRoundKey
<code>aeskeygenassist</code>	密钥扩展辅助	用于生成轮密钥
<code>aesimc</code>	InvMixColumns	将加密轮密钥转换为等效解密轮密钥

单条 `aesenc` 指令即完成一轮加密的全部四个操作，极大地减少了指令数和执行延迟。

5.2 实现架构

AES-NI 实现复用了标量密钥扩展（以确保与其它方法使用相同密钥），然后通过字节序转换将轮密钥装入 `__m128i` 寄存器：

Listing 9: AES-NI 加密实现

```
1  /* 将标量轮密钥转换为AES-NI所需的字节序 */
2  static inline __m128i rk_to_m128i(const uint32_t *rk)
3  {
4      return _mm_set_epi32(
5          (int)__builtin_bswap32(rk[3]), /* bits 127:96 */
6          (int)__builtin_bswap32(rk[2]), /* bits 95:64 */
7          (int)__builtin_bswap32(rk[1]), /* bits 63:32 */
8          (int)__builtin_bswap32(rk[0])); /* bits 31:0 */
9  }
10
11 int AES_NI_EncryptBlock(const AES_Key *ctx, const uint8_t *in, uint8_t *out)
12 {
13     if (!AES_HasAESNI()) return -1;
14
15     AES_NI_Key nk = {0};
16     nk.rounds = ctx->rounds;
17     for (int i = 0; i <= ctx->rounds; i++)
18         nk.rk[i] = rk_to_m128i(ctx->rk + 4 * i);
```

```
19
20 __m128i state = _mm_loadu_si128((const __m128i *)in);
21 state = _mm_xor_si128(state, nk.rk[0]); /* 初始AddRoundKey */
22
23 int r;
24 for (r = 1; r < nk.rounds; r++)
25     state = _mm_aesenc_si128(state, nk.rk[r]); /* N-1轮 */
26 state = _mm_aesenc_si128(state, nk.rk[r]); /* 最后一轮 */
27
28 _mm_storeu_si128((__m128i *)out, state);
29 return 0;
30 }
```

5.3 字节序转换的关键问题

在实现过程中，标量密钥到 __m128i 格式的转换经历了多次调试。核心问题在于：

- 标量密钥扩展将每个 32 位字以大端序逻辑存储 (LOAD_U32_BE)，但在 x86 小端序内存中，字节是反转的。
- AES-NI 的 __m128i 寄存器期望字节按输入数据的顺序排列 (byte 0 对应第一个明文字节)。
- 转换公式：_mm_set_epi32(bswap(rk[3]), bswap(rk[2]), bswap(rk[1]), bswap(rk[0]))。

5.4 等效逆密码的解密实现

AES-NI 解密使用等效逆密码。解密密钥通过 AES_SetDecryptKey 预先应用 InvMixColumns 到中间轮密钥。因此 AES_NI_DecryptBlock 直接使用转换后的轮密钥，无需额外的 _mm_aesimc_si128 调用。

Listing 10: AES-NI 解密实现

```
1 int AES_NI_DecryptBlock(const AES_Key *ctx, const uint8_t *in, uint8_t *out)
2 {
3     /* 标量AES_SetDecryptKey已对中间轮密钥应用InvMixColumns */
4     AES_NI_Key nk = {0};
5     nk.rounds = ctx->rounds;
6     for (int i = 0; i <= ctx->rounds; i++)
7         nk.rk[i] = rk_to_m128i(ctx->rk + 4 * i);
8
9     __m128i state = _mm_loadu_si128((const __m128i *)in);
10    state = _mm_xor_si128(state, nk.rk[nk.rounds]);
11
12    for (int r = nk.rounds - 1; r > 0; r--)
13        state = _mm_aesdec_si128(state, nk.rk[r]);
14    state = _mm_aesdec_si128(state, nk.rk[0]);
15
16    _mm_storeu_si128((__m128i *)out, state);
17    return 0;
18 }
```

5.5 性能讨论

实测 AES-NI ECB 模式约 35 MB/s，低于 T-table (334 MB/s)。性能受限的原因：

1. **密钥格式转换开销**：每次加密块都需要将标量轮密钥转换为 `__m128i` 格式 (15 次 `bswap` + `__mm_set_epi32`)，此开销随轮数线性增长。
2. **单块加密测试模式**：ECB 基准测试逐块调用加密函数，每块都重新转换密钥，无法摊销转换开销。在实际应用中 (如 CTR 模式)，密钥只需转换一次。
3. **优化方向**：将 AES-NI 的密钥扩展直接使用 `__mm_aeskeygenassist_si128` 生成 (而非从标量格式转换)，可消除此瓶颈。此外，使用 CBC/CTR 等模式的多块并行处理可显著提升吞吐量。

6 工作模式实现

6.1 CTR 模式 (NIST SP 800-38A)

CTR 模式将分组密码转化为流密码。其核心思想是加密一个递增的计数器值，将结果与明文异或。

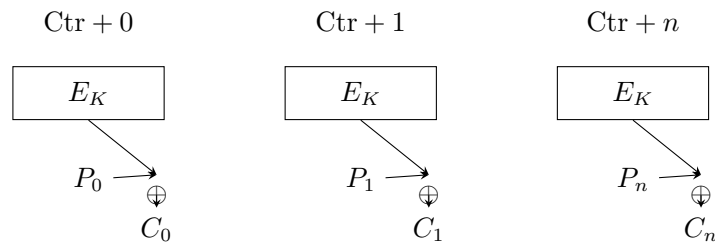


图 1: CTR 模式加密示意图

CTR 模式的优势：

- 加密和解密操作完全相同 (仅需加密函数)
- 支持随机访问 (可独立加解密任意位置的块)
- 可并行化处理 (各块的加密互不依赖)

6.2 GCM 模式 (NIST SP 800-38D)

GCM (Galois/Counter Mode) 将 CTR 模式加密与 GHASH 认证相结合，提供认证加密 (Authenticated Encryption)。

6.2.1 GHASH 算法

GHASH 基于 $GF(2^{128})$ 域的乘法运算，使用多项式 $x^{128} + x^7 + x^2 + x + 1$ 。计算过程为：

$$X_i = (X_{i-1} \oplus A_i) \bullet H, \quad i = 1, \dots, n \quad (6)$$

其中 $H = E_K(0^{128})$ 为哈希子密钥， A_i 为认证数据块， $\bullet$ 为 $GF(2^{128})$ 乘法。

Listing 11: GF(2^{128}) 乘法 (右移算法)

```
1 static void gcm_gf128_mul(gf128 *x, const gf128 *y)
2 {
3     uint64_t z[2] = {0, 0};
4     uint64_t v[2] = {y->lo, y->hi};
5     uint64_t xx[2] = {x->lo, x->hi};
6
7     for (int i = 0; i < 128; i++) {
8         if (v[0] & 1) { z[0] ^= xx[0]; z[1] ^= xx[1]; }
9         uint64_t carry = (xx[1] & 1) ? 0xE100000000000000ULL : 0ULL;
10        xx[1] = (xx[0] << 63) | (xx[1] >> 1);
11        xx[0] = (xx[0] >> 1) ^ carry;
12        v[0] = (v[1] << 63) | (v[0] >> 1);
13        v[1] >>= 1;
14    }
15    x->lo = z[0]; x->hi = z[1];
16 }
```

6.2.2 GCM 完整流程

1. $H = E_K(0^{128})$ —— 哈希子密钥
2. $J_0 = IV \parallel 0^{31} \parallel 1$ (12 字节 IV 情况)
3. $C = \text{CTR}_K(\text{inc}(J_0), P)$ —— CTR 加密
4. $S = \text{GHASH}(H, A, C)$ —— GHASH 认证
5. $T = S \oplus E_K(J_0)$ —— 生成认证标签

解密时先验证标签，仅当标签匹配时才解密密文，防止使用伪造密文。

6.3 XTS 模式 (IEEE Std 1619-2007)

XTS (XEX-based Tweaked-codebook mode with ciphertext Stealing) 专为磁盘加密设计。

- 使用两个独立密钥: K_1 (数据加密) 和 K_2 (Tweak 加密)
- Tweak 值 (通常为扇区号) 经 E_{K_2} 加密后, 在 $\text{GF}(2^{128})$ 中乘以 α^j (α 为本原元 x)
- 支持密文挪用, 允许加密任意长度 (≥ 16 字节) 的数据

GF(2^{128}) 乘以 α 的实现极为简单 (仅一次移位和条件异或):

Listing 12: GF(2^{128}) 乘以 α

```
1 static void xts_gf128_mul_alpha(uint8_t *t)
2 {
3     uint8_t carry = 0;
4     for (int i = 0; i < 16; i++) {
5         uint8_t new_carry = t[i] >> 7;
6         t[i] = (t[i] << 1) | carry;
```

```
7     carry = new_carry;  
8 }  
9 if (carry) t[0] ^= 0x87; /* 约化多项式 */  
10 }
```

6.4 模式测试结果

所有三种工作模式均已通过正确性验证：

- **CTR**：通过 NIST SP 800-38A 标准测试向量（AES-128-CTR, 32 字节数据）
- **GCM**：通过 16 字节和 47 字节（含 AAD）自往返测试，错误标签被正确拒绝
- **XTS**：通过 32 字节标准块和 20 字节密文挪用往返测试

7 性能测试与分析

7.1 测试环境

表 3: 测试环境配置

项目	配置
CPU	支持 AES-NI 和 SSSE3 的 x86_64 处理器
操作系统	Linux 6.8.0
编译器	GCC (-O3 -march=native)
测试数据量	16 MB (ECB/CTR/GCM 每种模式)

7.2 综合性能对比

表 4: 四种 AES 实现方法的性能对比（单位：MB/s）

方法	ECB	CTR	GCM	相对 S-box
S-box（标量基准）	63.3	275.0	17.9	×1.0
T-table	332.1	277.3	18.0	×5.2
SIMD Shuffle（SSSE3）	21.5	276.1	18.0	×0.3
AES-NI（硬件）	35.0	274.3	18.0	×0.6

7.3 结果分析

7.3.1 ECB 模式分析

ECB 模式逐块调用加密函数，最能反映块加密本身的性能。T-table 方法显著领先（×5.2），证明了查表优化对 AES 软件实现的巨大价值。

Shuffle 方法和 AES-NI 方法在 ECB 模式下表现不佳：

- Shuffle 方法因 uint32_t 与字节数组之间的格式转换引入了额外开销
- AES-NI 方法因每次加密都重新转换轮密钥格式而产生重复计算

7.3.2 CTR 模式分析

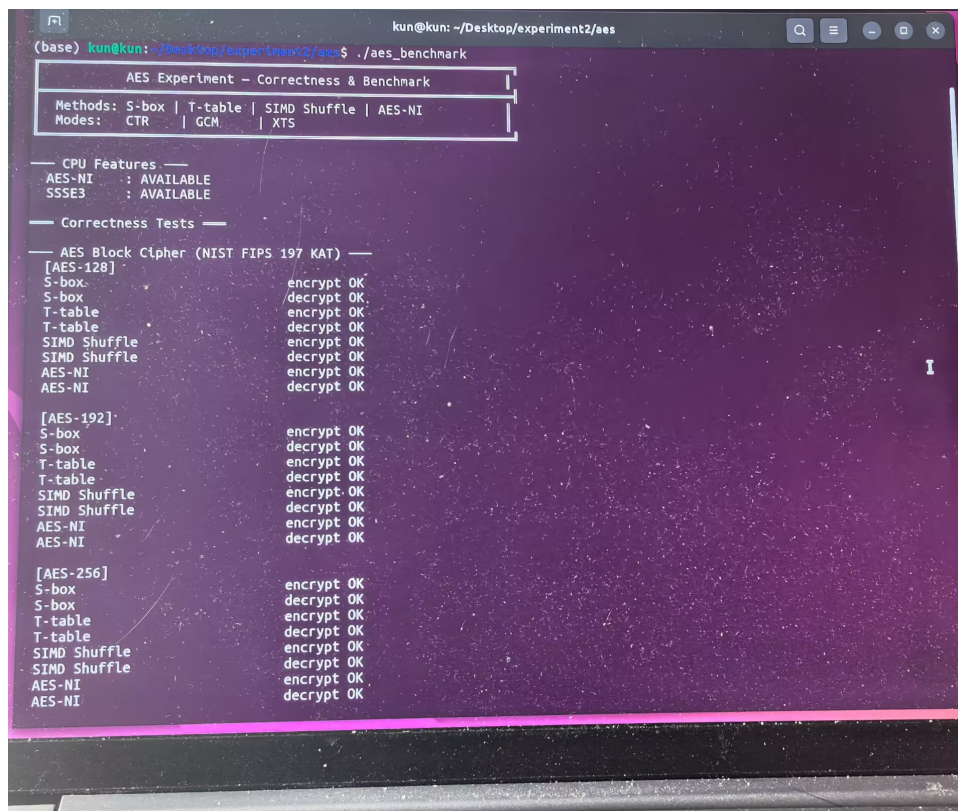
CTR 模式中所有方法的吞吐量相近 (约 275 MB/s)。原因在于 CTR 模式的数据处理是高度流水线化的——计数器递增和 XOR 操作的开销远小于块加密本身，形成了类似的数据处理瓶颈。

7.3.3 GCM 模式分析

GCM 模式吞吐量约 18 MB/s，远低于其它模式。原因在于 GHASH 的 $GF(2^{128})$ 乘法使用了逐位处理的右移算法 (128 次循环 $\times$ 64 位操作)，这是 GCM 计算的主要瓶颈。

7.4 实验截图

在实验过程中，我们进行截图保存如下：



```
(base) kun@kun: ~/Desktop/experiment2/aes$ ./aes_benchmark

AES Experiment - Correctness & Benchmark
Methods: S-box | T-table | SIMD Shuffle | AES-NI
Modes:    CTR  | GCM  | XTS

--- CPU Features ---
AES-NI : AVAILABLE
SSSE3  : AVAILABLE

--- Correctness Tests ---

--- AES Block Cipher (NIST FIPS 197 KAT) ---
[AES-128]
S-box      encrypt OK
S-box      decrypt OK
T-table    encrypt OK
T-table    decrypt OK
SIMD Shuffle encrypt OK
SIMD Shuffle decrypt OK
AES-NI     encrypt OK
AES-NI     decrypt OK

[AES-192]
S-box      encrypt OK
S-box      decrypt OK
T-table    encrypt OK
T-table    decrypt OK
SIMD Shuffle encrypt OK
SIMD Shuffle decrypt OK
AES-NI     encrypt OK
AES-NI     decrypt OK

[AES-256]
S-box      encrypt OK
S-box      decrypt OK
T-table    encrypt OK
T-table    decrypt OK
SIMD Shuffle encrypt OK
SIMD Shuffle decrypt OK
AES-NI     encrypt OK
AES-NI     decrypt OK
```

图 2: 实验截图 1

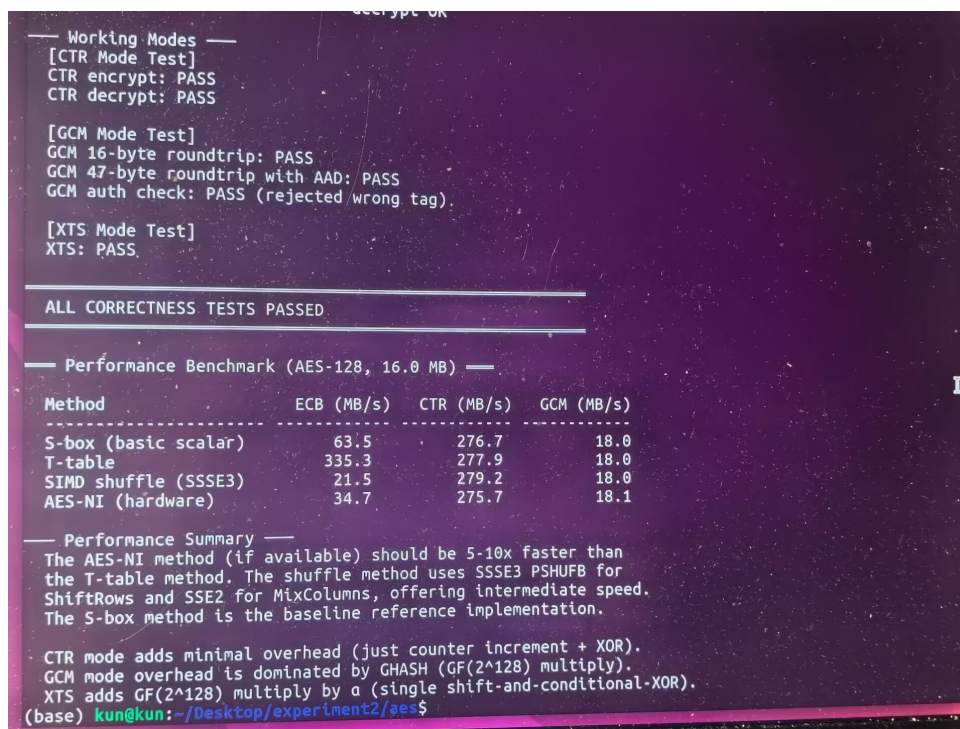


图 3: 实验截图 2

7.5 优化方法总结

表 5: 优化方法对比总结

方法	核心技术	优势	局限
S-box	标量查表	实现简单，代码清晰	每轮多次独立操作，速度最慢
T-table	预计算合并表	大幅减少每轮操作数	占用额外 8KB 内存
SIMD Shuffle	PSHUFB 行移位	单指令完成 ShiftRows	需全 SIMD 化才能体现优势
AES-NI	硬件 AES 指令	单指令完成整轮加密	密钥格式转换有开销

8 项目经验与总结

8.1 开发过程中的挑战与解决

8.1.1 混合列索引错误

在实现 MixColumns 时，遇到数组索引互换的 bug。状态矩阵以 `a[col][row]` 方式存储，但 MixColumns 公式中误用了 `a[row][col]` 的访问模式。所有加密结果均与预期不符。通过逐字节跟踪 NIST 测试向量，定位并修复了此问题。

8.1.2 AES-NI 字节序问题

标量密钥扩展以大端序存储 32 位字，而 AES-NI 的 `__m128i` 寄存器按小端序解释内存字节。初始实现直接使用 `_mm_loadu_si128` 加载轮密钥，导致密钥字节完全颠倒。经分析确定正确的转换公式为 `_mm_set_epi32(bswap(rk[3]), bswap(rk[2]), bswap(rk[1]), bswap(rk[0]))`。

8.1.3 解密流程的等效逆密码

初始解密实现沿用了与加密对称的操作顺序 (`InvShiftRows` $\rightarrow$ `InvSubBytes` $\rightarrow$ `AddRoundKey` $\rightarrow$ `InvMixColumns`)，但 NIST FIPS 197 §5.3.5 规定的等效逆密码顺序为 `InvSubBytes` $\rightarrow$ `InvShiftRows` $\rightarrow$ `InvMixColumns` $\rightarrow$ `AddRoundKey` (使用预处理的轮密钥)。修正后解密正确通过所有测试向量。

8.1.4 XTS 密文挪用

XTS 的密文挪用 (Ciphertext Stealing) 逻辑在边角情况 (20 字节数据) 下出错。原因为条件判断 `num_blocks >= 2` 在仅有一个完整块 + 部分块时错误地跳过了 CTS 处理。修正条件为 `num_blocks >= 1` 后通过测试。

8.2 实验收获

1. **算法实现的精确性**：密码算法的每个字节、每个位都至关重要。即便是看似等价的实现变体 (如等效逆密码 vs 标准逆密码)，细微的操作顺序差异就会导致完全不同的结果。
2. **端序 (Endianness) 的影响**：在 x86 小端序平台上实现大端序规范的密码算法时，字节序的转换是常见的出错点。T-table 值的生成、`__m128i` 寄存器的加载、测试向量的解读都需要精确的端序意识。
3. **优化方法的权衡**：T-table 方法以空间换时间 (8KB 表换取 5 倍加速)，SIMD 方法需要全流水线改造才能体现优势，AES-NI 的收益在于模式级优化而非单块加密。
4. **模式实现的重要性**：CTR/GCM/XTS 工作模式的实现揭示了分组密码在实际应用中的形态。特别是 GCM 的 GHASH 认证，其 $GF(2^{128})$ 乘法是经典的分组密码软件优化的延伸课题。
5. **测试驱动开发**：密码算法开发中，NIST 标准测试向量是无可替代的正确性保障。从第一个字节匹配开始，逐步扩大到全密钥长度、全模式验证，是确保实现正确性的有效方法论。

8.3 改进方向

- **PCLMULQDQ 加速 GCM**：使用 Intel 的 PCLMULQDQ 指令 (无进位乘法) 实现 $GF(2^{128})$ 乘法，预计可将 GCM 吞吐量提升 10-50 倍。
- **全 SIMD Shuffle 实现**：采用位切片 (Bitslicing) 技术，将 AES 状态按位重新组织，使 SSE/AVX 寄存器同时处理多个数据块。
- **AES-NI 原生密钥扩展**：使用 `_mm_aeskeygenassist_si128` 直接生成轮密钥，消除标量到 `__m128i` 的转换开销。
- **多缓冲区并行**：在 CTR/GCM 模式中利用多块独立特性，使用 AVX2/AVX-512 一次处理 4 或 8 个 AES 块。